

Техническое задание

на разработку прошивки для модуля Hulyн WT32-ETH01 V1.4 (ESP3...SP-IDF)
и лабораторного WRT-стенда на базе OpenWrt/VirtualBox

1. Общие сведения

1.1. Основание для разработки

Настоящее техническое задание (ТЗ) устанавливает требования к:
прошивке для модуля Hulyн WT32-ETH01/02 V1.4 на базе ESP32, р... с использованием
ESP-IDF (без применения Arduino-фреймворка);
лабораторному стенду на базе OpenWrt (далее — WRT-стенд), раз...иртуальной
машине под Oracle VirtualBox на рабочем компьютере.

1.2. Цель разработки

Цели разработки:

обеспечить непрерывное высокочастотное сканирование Bluetooth...ь минимально
возможное по техническим характеристикам модуля);
реализовать сбор и первичную обработку событий BLE и Wi-Fi, их буферизацию,
логирование и отправку по сети;
подготовить формат обмена на базе Protocol Buffers (Protobuf)...анных в последующие
компоненты системы (в т.ч. БД PostgreSQL);
создать лабораторный WRT-стенд для отработки сценариев интегр... маршрутизатор
→ подсистема приёма данных» в тестовом контуре.
На первом этапе WRT-стенд рассматривается как учебно-лаборато...ботки решений, а
не как окончательная промышленная реализация.

1.3. Область применения

Разрабатываемый комплекс предназначен для использования в системах:
сбора телеметрического радиоокружения (BLE/Wi-Fi);
задач позиционирования и построения радиокарт.

2. Нормативные ссылки и используемые документы

При разработке необходимо использовать:

официальную документацию ESP-IDF (актуальная LTS-версия);
документацию по ESP32 (техническое описание, TRM);
документацию на модуль Hylin WT32-ETH01 V1.4 (схема, описание выводов, питание);
документацию на Ethernet PHY;
документацию на OpenWrt и OpenWrt SDK (для сборки пакетов);
документацию на PostgreSQL (актуальная в контуре эксплуатации);

3. Термины, обозначения и сокращения

BLE — Bluetooth Low Energy.

Wi-Fi — беспроводной интерфейс IEEE 802.11.

ESP-IDF — SDK/фреймворк Espressif для разработки под ESP32.

Protobuf — Protocol Buffers, формат описания и сериализации данных.

Модуль — Hylin WT32-ETH01/02 V1.4 на базе ESP32 и Ethernet PHY.

WRT-стенд — виртуальная машина OpenWrt под Oracle VirtualBox, применяемая как
тестовый стенд.

Сервер приёма — программный компонент, принимающий данные от модуля и выполняющий их запись (в лог/БД).

Событие сканирования — запись о зафиксированном радиисточнике (BLE или Wi-Fi).

4. Объект разработки и архитектура

4.1. Аппаратная платформа

Модуль Hulyн WT32-ETH01/02 V1.4:

MCU: ESP32 с поддержкой Wi-Fi и BLE;

Ethernet: PHY (LAN8720 или аналог), интерфейс RMII;

интерфейсы: UART, Ethernet, Wi-Fi, BLE, GPIO и др. (по документации на модуль);

питание: согласно паспорту изделия.

4.2. Схема взаимодействия

Базовая топология:

Модуль подключается по Ethernet к локальной сети.

В сети развёрнут WRT-стенд (OpenWrt VM), способный принимать данные от модуля, либо выполняется прямое подключение модуля к тестовому серверу.

Данные событий BLE/Wi-Fi передаются с модуля по сети (TCP/UDP/HTTP, см. разд. 6) в бинарном формате (целевой — Protobuf) для последующей записи и анализа.

4.3. Логическая архитектура прошивки

В прошивке должны быть реализованы, как минимум, следующие логические компоненты:

BLE Scanner — подсистема сканирования BLE (инициализация, запуск, получение результатов, формирование событий);

Wi-Fi Scanner — подсистема сканирования Wi-Fi;

Event Buffer — буферизация событий BLE/Wi-Fi в оперативной памяти и/или во внутреннем хранилище;

Storage/Logging — модуль структурированного логирования на локальное хранилище (SPIFFS/LittleFS/NVS);

Network Transport — модуль установления сетевого соединения и передачи данных (в т.ч. Protobuf-батчей);

Config/Management — механизм конфигурации (NVS, интерфейс управления, сброс к настройкам по умолчанию);

Monitoring/Watchdog — контроль состояния задач и перезапуск при критических сбоях.

5. Функциональные требования к прошивке

5.1. Режимы работы

Прошивка должна обеспечивать:

штатный режим:

непрерывное сканирование BLE и/или сканирование Wi-Fi;

отправку данных при наличии связи;

логирование событий по заданной политике (политику предлагает разработчик);

режим отладки (повышенная подробность логирования по UART/внутренний лог);

режим восстановления конфигурации (сброс к заводским параметрам).

5.2. Сканирование BLE

Требования:

Период/частота:

обеспечивать временное разрешение по результатам сканирования...уже 100 мс и/или минимально допустимый программируемый модуль;

допускается использование непрерывного сканирования с периодической выборкой результатов каждые ≤ 100 мс.

Основные характеристики события BLE (до сериализации):

время обнаружения, включающее:

– монотонную метку `event_monotonic_ms` — время в миллисекундах от момента запуска устройства по монотонному таймеру (без обратных ходов и скачков);

– абсолютную метку `event_utc_ms` — время обнаружения в миллисекундах в формате Unix time (UTC), вычисляемое на основании монотонного времени и параметров синхронизации (см. п. 5.6).

MAC-адрес устройства;

тип адреса (`public/random`, если доступно в API ESP-IDF);

RSSI, дБм;

данные рекламного пакета (опционально усечённые до разумной длины);

при наличии — номер канала и дополнительные поля из API ESP-IDF.

Поведение под нагрузкой:

при количестве видимых устройств до целевого уровня (значени...я устойчивым, без существенного увеличения интервала обработки.

5.3. Сканирование Wi-Fi

Требования:

Периодичность:

задается максимально возможным для используемого модуля;

сканирование Wi-Fi не должно приводить к ухудшению характери...одится последовательно с задаваемым в прошивке характеристикам.

Основные поля события Wi-Fi:

время обнаружения;

BSSID;

SSID;

RSSI;

канал;

тип авторизации/шифрования (по `enum ESP-IDF`).

События Wi-Fi должны включаться в общий поток обрабатываемых и передаваемых событий.

5.4. Буферизация и логирование

Буферизация:

события BLE/Wi-Fi должны складываться в буфер (очередь) для последующей передачи либо записи в файл;

при недоступности сети должна быть обеспечена запись в лока...ерпания выделенного объёма (технология предлагает разработчик).

Логирование:

структурированное логирование событий (CSV/JSON/бинарный фо...ат — по выбору исполнителя, с приоритетом на простоту анализа);

ротация логов по размеру/количеству файлов;

системный лог (ошибки, перезапуски, сетевые события) — в UART и/или отдельный файл.

5.5. Передача данных по сети

Транспорт:

поддержка как минимум одного из вариантов (при возможности реализовать оба):

TCP-клиент (постоянное подключение к заданному хосту и порту);

или HTTP-клиент (POST-запросы на заданный URI).

Формирование пакетов:

события собираются в батчи по количеству и/или по времени (...ий в батче, не реже одного батча за заданный интервал времени);

тело сетевого сообщения содержит сериализованный Protobuf-батч (см. разд. 6).

Поведение при ошибках:

при невозможности установления соединения — переход в режим накопления с периодическими попытками переподключения;

при частичных ошибках передачи — регистрация ошибок в системном логге.

5.6. Синхронизация времени и временные метки

В прошивке должен быть реализован монотонный таймер, обеспечивающий счёт времени в миллисекундах от момента запуска устройства без обратных ходов и скачков (далее — монотонное время).

Каждое событие BLE/Wi-Fi должно содержать:

– поле `event_monotonic_ms` — время обнаружения в миллисекундах по монотонному таймеру;

– поле `event_utc_ms` — абсолютное время обнаружения в миллисекундах в формате Unix time (UTC), получаемое по формуле $\text{event_utc_ms} = \text{event_monotonic_ms} + \text{offset_ms}$, где `offset_ms` — смещение между монотонным временем устройства и UTC, определяемое при синхронизации.

В прошивке должна быть реализована синхронизация системного времени по одному или нескольким источникам:

– SNTP/NTP через сеть (обязательный вариант на текущем этапе, источник — WRT-стенд или иной NTP-сервер в локальной сети);

– в дальнейшем допускается использование GPS в качестве источника времени (через маршрутизатор или напрямую).

В момент успешной синхронизации времени модуль должен:

– считать текущее абсолютное время `utc_ms_sync` (Unix time в миллисекундах, UTC) из выбранного источника;

– считать текущее монотонное время `mono_ms_sync` из внутреннего таймера;

– вычислить и сохранить смещение $\text{offset_ms} = \text{utc_ms_sync} - \text{mono_ms_sync}$;

– пометить состояние времени как синхронизированное (`time_synced = true`).

До первой успешной синхронизации времени `event_monotonic_ms` заполняется и используется для упорядочивания событий, а `event_utc_ms` может принимать значение 0 (или иное согласованное служебное значение); при этом в батчах/записях должен присутствовать признак `time_synced = false` или аналогичный индикатор отсутствия синхронизации.

После получения новой синхронизации времени при малой корректировке (порог определяется в прошивке, например $\leq 50\text{--}100$ мс) допускается плавная корректировка `offset_ms`; при значительной разнице времени должна фиксироваться запись в

системном логге о пересинхронизации с указанием старого и нового значения смещения.

Требуемая точность абсолютной временной метки `event_utc_ms` для целей последующей корреляции с данными GPS и другими источниками составляет не хуже ± 50 мс при нормальной работе сети и источника времени; допустимое максимальное отклонение — не хуже ± 100 мс.

6. Форматы данных и Protobuf

6.1. Общие требования

Для обмена между модулем и принимающей стороной должен быть использован формат Protocol Buffers (Protobuf, версия `proto3`).

Состав и структура Protobuf-сообщений должны быть разработаны исполнителем на основании:

перечня параметров BLE/Wi-Fi событий (раздел 5);

требований к последующей обработке и хранению (в т.ч. в БД PostgreSQL);

требований к расширяемости (возможность добавления новых полей без нарушения совместимости).

Исполнитель обязан

выполнить анализ необходимого состава полей, типов и группировок;

подготовить один или несколько `.proto`-файлов, задокументировать выбранную структуру;

согласовать схему с заказчиком.

6.2. Пример (иллюстративный, не обязательный)

Пример упрощённой схемы, иллюстрирующий ожидаемый уровень детализации (структура и номера полей являются примером, а не готовым стандартом):

```
message BleEvent {
  uint64 timestamp_ms = 1;
  string addr          = 2;
  int32  rssi_dbm     = 3;
  bytes  adv_data     = 4;
}
```

```
message WifiEvent {
  uint64 timestamp_ms = 1;
  string bssid        = 2;
  string ssid         = 3;
  int32  rssi_dbm     = 4;
}
```

```
message DeviceInfo {
  string device_id  = 1;
  string fw_version = 2;
}
```

```
message ScanBatch {
  DeviceInfo device = 1;
```

```
repeated BleEvent ble    = 2;  
repeated WifiEvent wifi  = 3;  
uint64 batch_seq        = 4;  
}
```

Этот пример служит только для ориентира по уровню абстракции. Реальная схема должна быть разработана исполнителем и оформлена отдельным документом (описание полей, версия, правила расширения).

7. Нефункциональные требования

7.1. Производительность

обеспечивать регистрацию BLE-событий с временным шагом не хуже 100 мс при типовой нагрузке;
обеспечивать стабильную работу при длительном непрерывном функционировании (сутки и более).

7.2. Надёжность и устойчивость

использовать штатные средства ESP-IDF для обработки критических ошибок (watchdog, panic-перезапуск);
обеспечить восстановление работы после кратковременных сбоев связи без вмешательства оператора;
предусмотреть поведение при заполнении локального хранилища (остановка логирования, сигнализация в логах).

7.3. Ресурсы

выполнить оценку использования RAM и Flash;
спроектировать разметку Flash (раздел прошивки, NVS, файловая система под логи) с указанием объёмов;
обеспечить работу в типичной конфигурации ESP32 без дефицита памяти.

7.4. Информационная безопасность (минимальный уровень)

ограничить возможности удалённого управления только необходимыми командами;
исключить наличие жёстко зашитых чувствительных данных (пароли, ключи) в открытом виде;
заложить возможность дальнейшей интеграции транспортного шифрования (TLS) без радикальной переработки архитектуры.

8. WRT-стенд на базе OpenWrt/VirtualBox

8.1. Назначение

WRT-стенд служит для:

изучения и отработки схемы взаимодействия модуля с маршрутизатором;
отладки приёма Protobuf-пакетов, их базовой обработки и переотправки;
подготовки к дальнейшей интеграции с реальными устройствами IRZ.

На данном этапе стенд носит лабораторный характер и не предназначен для прямого промышленного применения.

8.2. Требования к WRT-стенду

Среда исполнения:

Oracle VM VirtualBox под ОС Windows;

гостевая ОС — OpenWrt (образ согласованной версии).

Сетевые настройки:

как минимум один сетевой интерфейс в режиме Bridge для доступа из локальной сети и взаимодействия с модулем;

при необходимости — дополнительный NAT-интерфейс для доступа к Интернет.

Документация:

описание процесса развёртывания WRT-стенда (получение образа, создание VM, настройка сети);

фиксация используемой конфигурации (версии, параметры сети).

8.3. Функциональные требования к программной части на WRT-стенде

На WRT-стенде требуется реализовать тестовый приёмник:

слушающий указанный TCP-порт или HTTP-endpoint;

принимающий бинарные Protobuf-пакеты (ScanBatch или аналог);

выполняющий:

либо запись принятых пакетов в файл (сырой бинарный или JSON-представление);

либо (опционально) пересылку в внешний компонент, который осуществляет запись в PostgreSQL.

Допускаются упрощения:

логика обработки может быть реализована в виде простого демона/сервиса на

C/C++/Python/Lua и т.п., запущенного вручную или через init-скрипт;

полный цикл создания «правильного» OpenWrt-пакета (с Makefile и автозапуском) на первом этапе не является обязательным, но приветствуется как вариант развития.

Основное требование: стенд должен позволять воспроизводимо:

поднять OpenWrt-VM;

запустить приёмник;

принять Protobuf-данные от модуля и зафиксировать их в удобном для анализа виде.

9. Среда разработки

9.1. Для прошивки ESP32

ESP-IDF (актуальная LTS-версия, фиксируется в отдельном документе или README);

ОС разработчика: Linux или Windows с WSL2;

установка стандартного набора инструментов ESP-IDF (Python, CMake, Ninja и др.);

структура проекта — стандартная для ESP-IDF (директория main, при необходимости components и др.).

9.2. Для WRT-стенда

Oracle VM VirtualBox;

образ OpenWrt согласованной версии;

по необходимости — OpenWrt SDK для экспериментов с пакетами.

10. Тестирование и приёмка

10.1. Виды испытаний

Функциональные испытания:

корректность BLE/Wi-Fi-сканирования;

формирование и отправка данных;

работа логирования и буферизации.

Интеграционные испытания:

передача Protobuf-пакетов с модуля на WRT-стенд;

проверка целостности и разбор принятых пакетов.

Длительные испытания:

работа в непрерывном режиме (не менее 8–24 часов) с контролем устойчивости.

10.2. Критерии приёмки

модуль обеспечивает требуемую частоту BLE-сканирования и формирует события с заданным набором параметров;

события BLE и Wi-Fi поступают в сетевой приёмник в виде Protobuf-батчей;

при временной недоступности сети события сохраняются в локальном логе до

исчерпания выделенного ресурса;

WRT-стенд позволяет воспроизводимо принимать и фиксировать Protobuf-сообщения для дальнейшего анализа.

11. Документация

По результатам работ должны быть предоставлены:

описание архитектуры прошивки и WRT-стенда (структура задач, взаимодействие компонентов);

описание форматов данных и разработанных .proto-файлов;

инструкция по сборке и прошивке модуля (ESP-IDF, команды и параметры);

инструкция по развёртыванию WRT-стенда и запуску приёмника;

краткий отчёт по испытаниям (с описанием сценариев и результатами).